Assignment 1: Using Git and Project Management Tools

Module: Applied Robotics and AI Project
Student: Kenzo Ji
Project: AI Flashcard Maker, a command-line tool that turns notes into flashcards with GPT-4o-mini
GitHub repository: https://github.com/kennnyyyyyyy/ai-flashcard-maker
Trello board: https://trello.com/b/lzUqMxdc

Contents

1. Where each part of the brief is covered
2. Setting up the Trello board
3. Setting up the GitHub repository
4. Add, edit, commit, push and clone
5. The difference between add, commit and push
6. Branches and pull requests
7. How Trello and GitHub are connected
8. Evidence of continuous use
9. Summary

1. Where each part of the brief is covered

Brief / marking criterion	Where to find the evidence
Set up and populate a project management board, invite the lecturer	Section 2, screenshots 1 to 5
Set up a Git repository and invite the lecturer	Section 3, screenshots 6 to 10
Add, Edit, Commit, Push and Clone	Section 4, screenshots 11 to 16
Discussion of the differences between add, commit and push	Section 5
Branch the repository, branches and PRs used properly	Section 6, screenshots 17 to 27
Integrated with GitHub	Section 7
Evidence of continuous and proper use: planning, WIP limits, retrospective, cards archived in Done with PR links	Section 8, screenshots 28 to 38

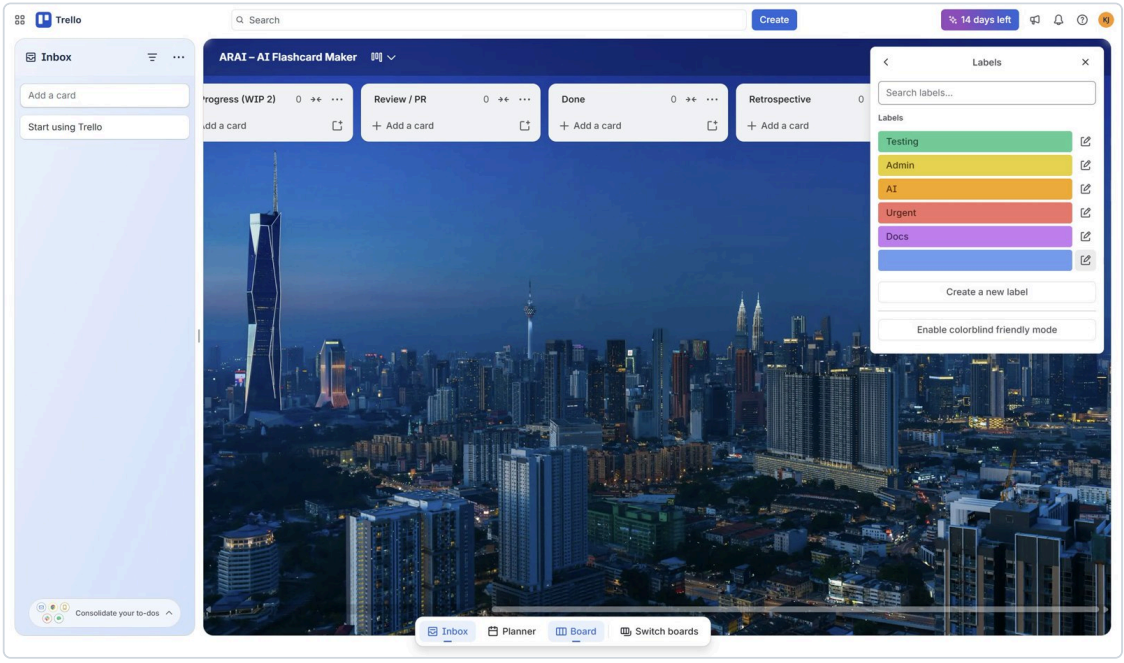
2. Setting up the Trello board

I created a board called **ARAI - AI Flashcard Maker** with six lists that follow my workflow from left to right:

Backlog -> To Do -> In Progress (WIP 2) -> Review / PR -> Done -> Retrospective

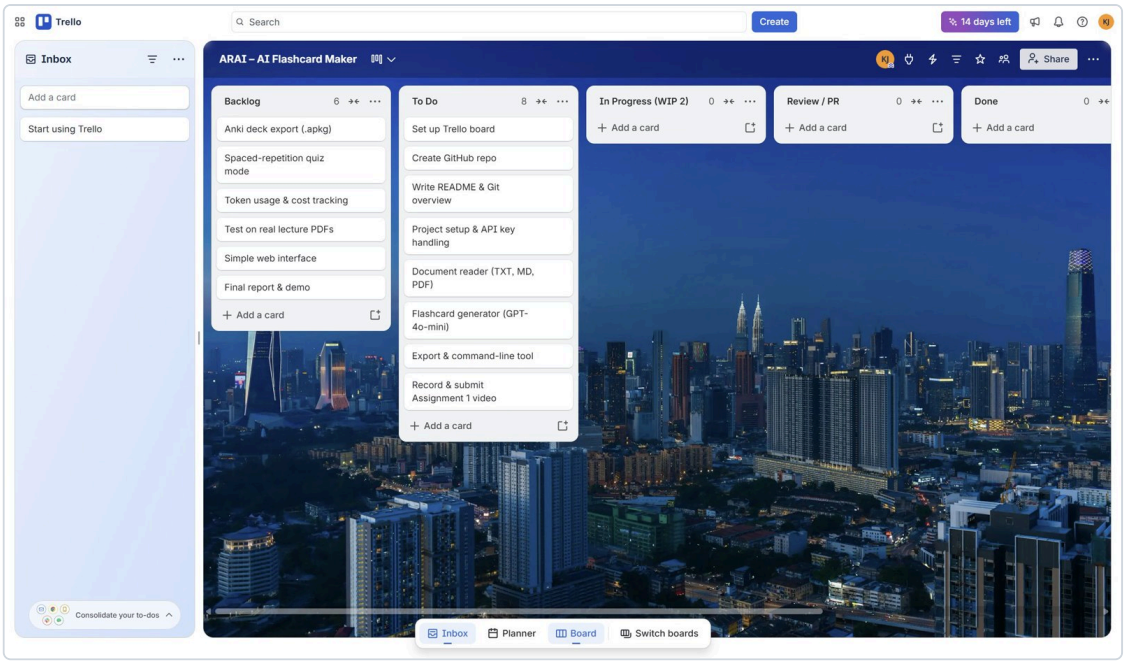
- **Backlog** holds work planned for later sprints, each with a later due date.
- **To Do** holds the current sprint.
- **In Progress** has a work-in-progress limit of 2, so I never have more than two tasks open at once.
- **Review / PR** holds cards whose pull request is open and waiting for tests.
- **Done** holds finished cards, and **Retrospective** holds the end-of-sprint review.

I named the six labels by type of work: Feature, AI, Testing, Docs, Admin and Urgent.



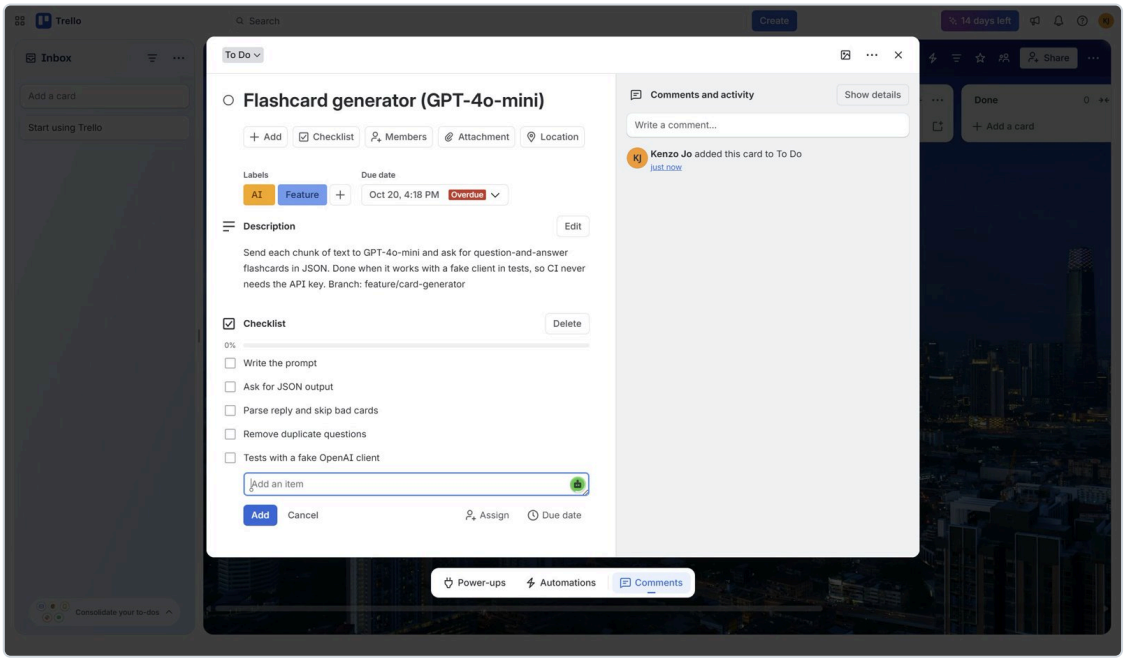
Screenshot 1: the lists and the named labels.

Every task is a card. The current sprint went into To Do and later ideas went into the Backlog.

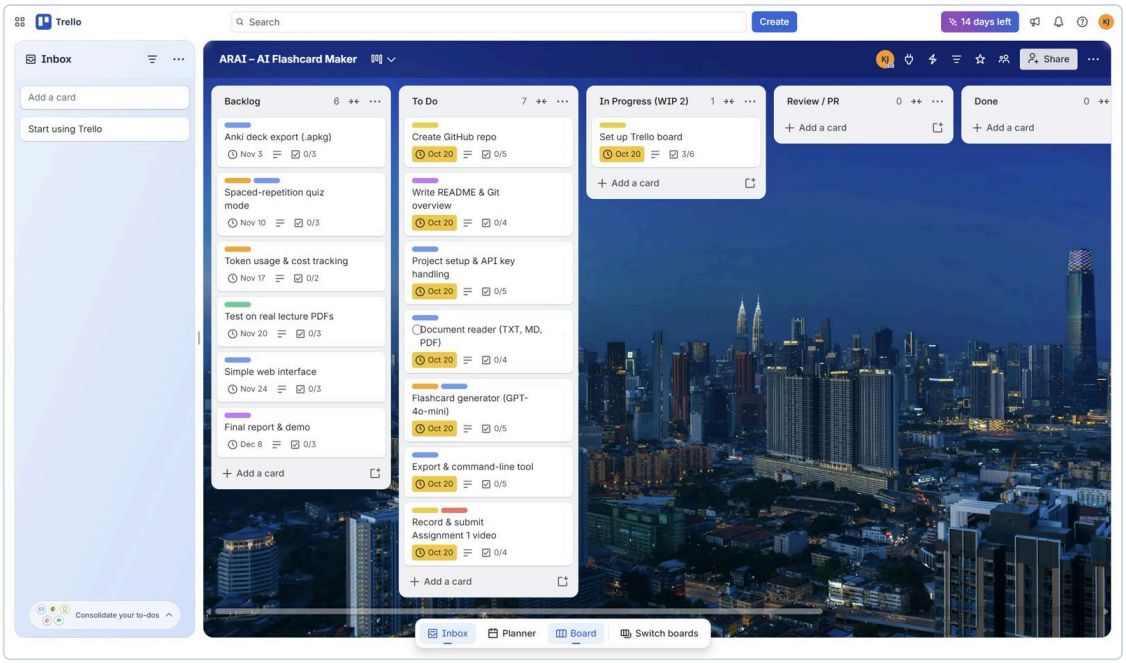


Screenshot 2: the cards for this sprint and the Backlog.

Each card has labels, a due date, a description of what "done" means, the Git branch it will use, and a checklist that breaks the task into steps.

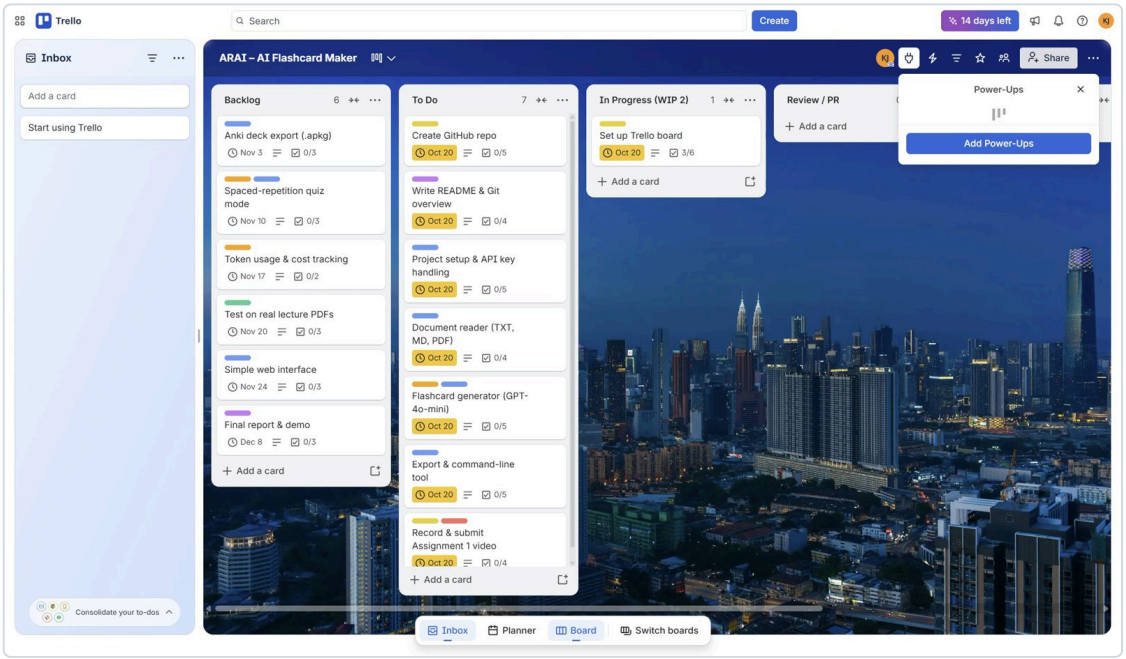


Screenshot 3: the Flashcard generator card with labels, due date, description, branch and checklist.



Screenshot 4: the populated board, with "Set up Trello board" in progress.

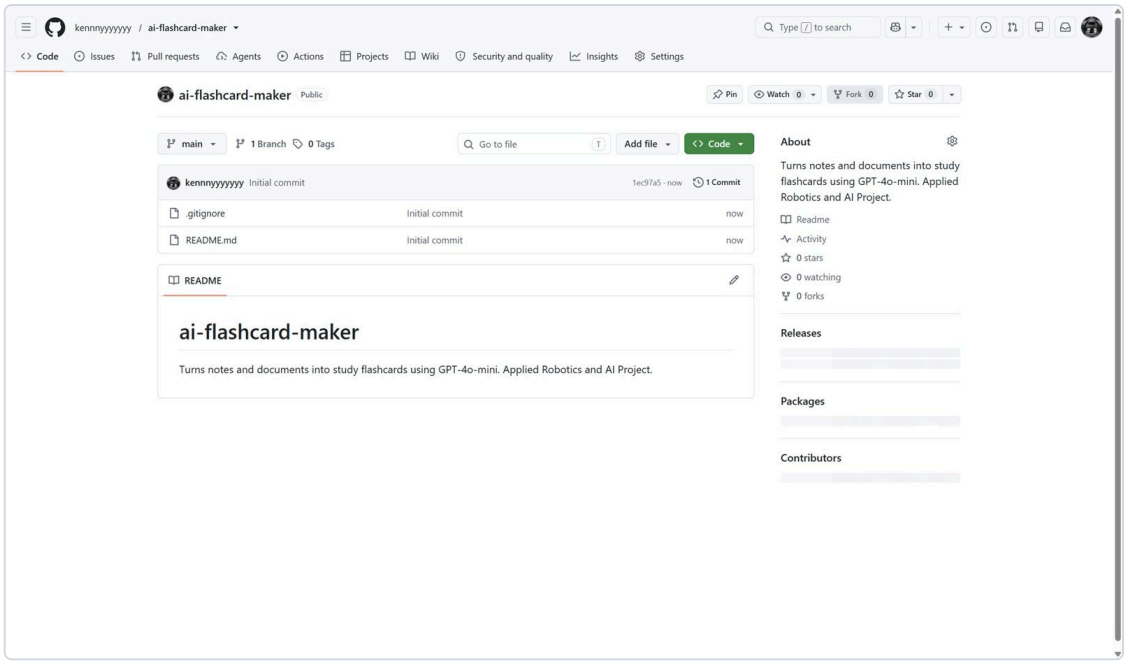
I added the **GitHub Power-Up**, which lets pull requests be attached to cards, and shared the board with my lecturer (sarina.tajudin@psb-academy.edu.sg), who is now a board member.



Screenshot 5: opening Power-Ups to add GitHub. Screenshot 24 shows it working, with a pull request attached to a card.

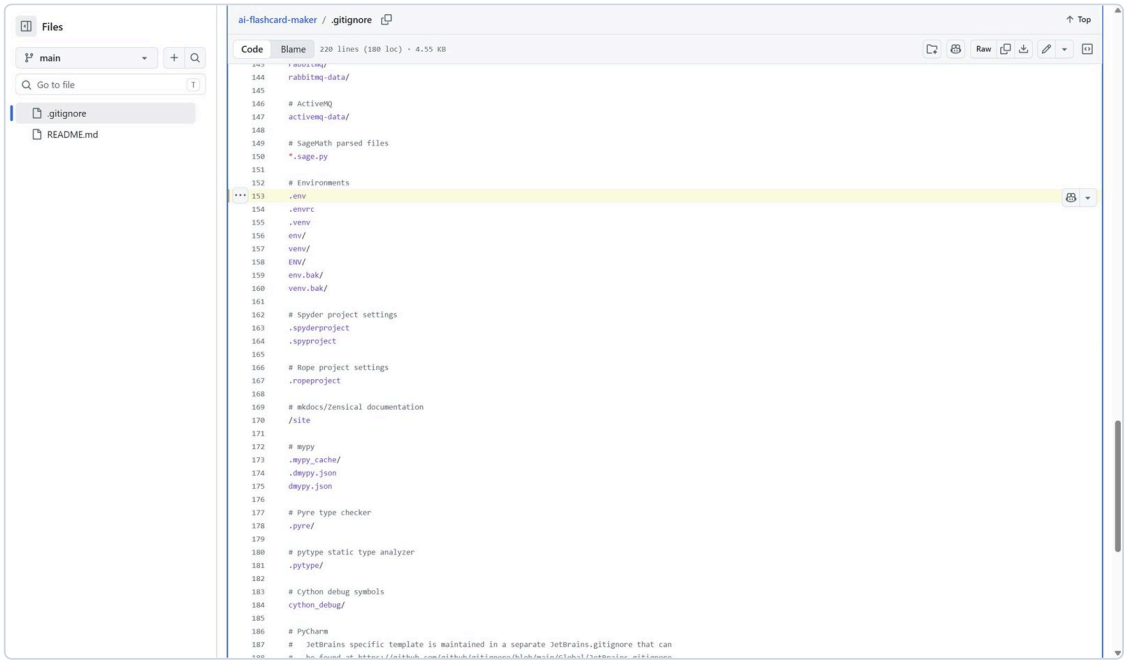
3. Setting up the GitHub repository

I created a public repository called **ai-flashcard-maker** with a README and the Python `.gitignore` template.



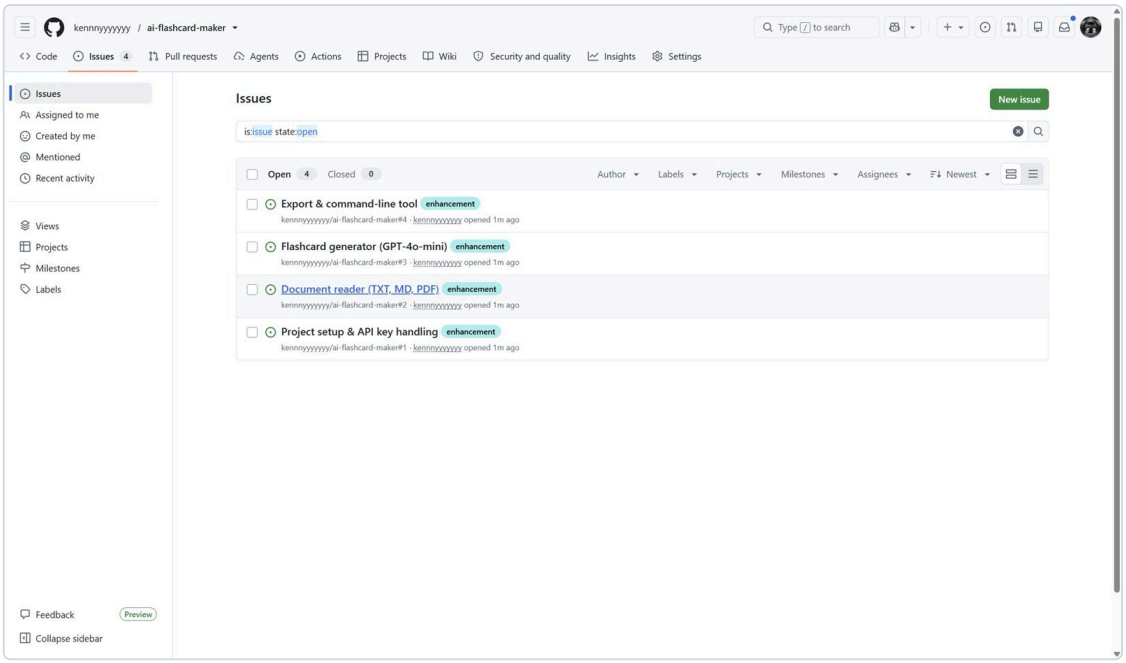
Screenshot 6: creating the public repository with a README and the Python .gitignore.

The `.gitignore` matters for this project because it already lists `.env`. My OpenAI API key lives in `.env`, so Git can never upload it.



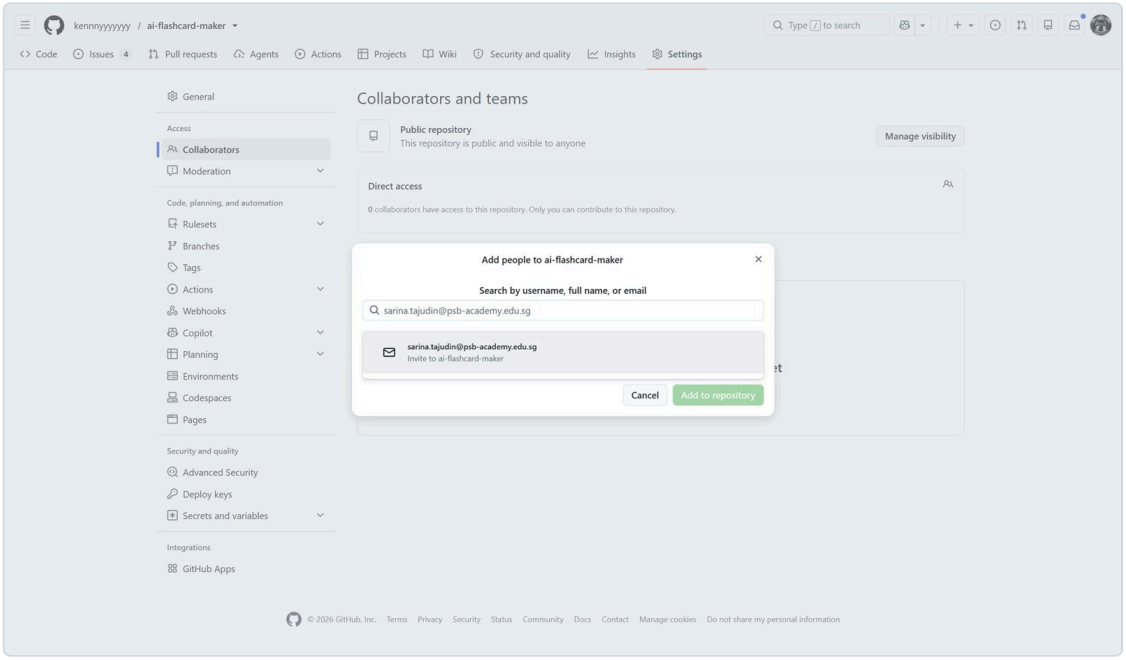
Screenshot 7: line 153 of .gitignore ignores the .env file.

I created one GitHub issue per feature. Each issue links to its Trello card and names its branch, and each pull request closes its issue when it is merged.



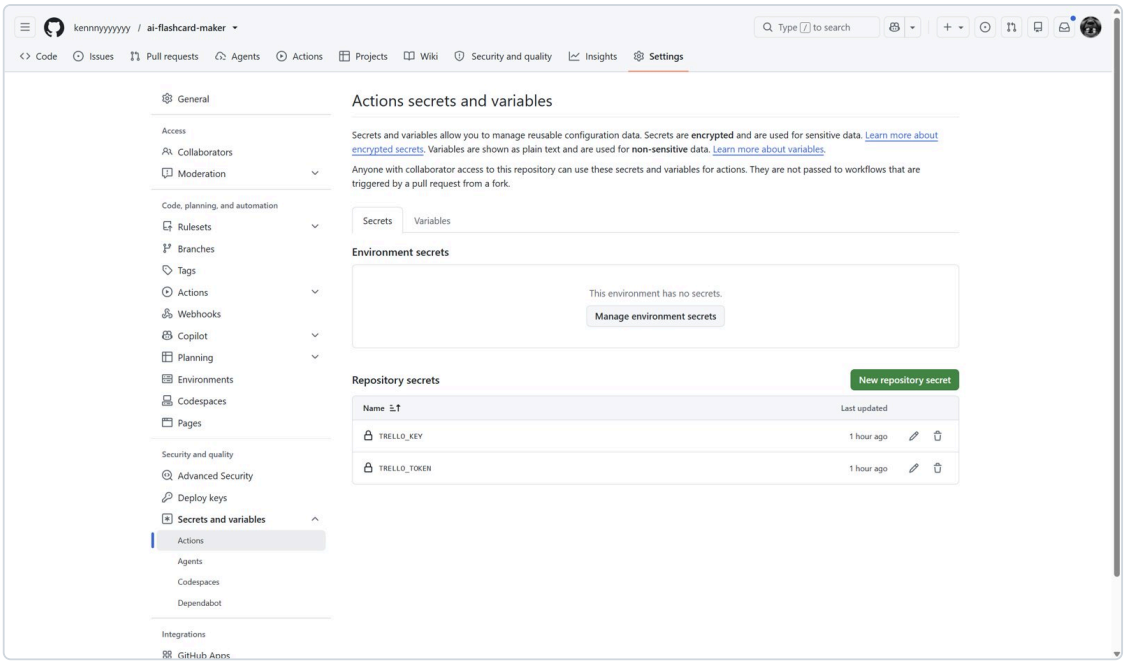
Screenshot 8: the four feature issues, each linked to a Trello card.

I invited my lecturer as a collaborator, so every commit, issue and pull request can be seen.



Screenshot 9: adding the lecturer as a collaborator.

To connect GitHub to Trello automatically, I created a Trello API key and token and stored them as encrypted **repository secrets**. Only the names are visible, never the values.

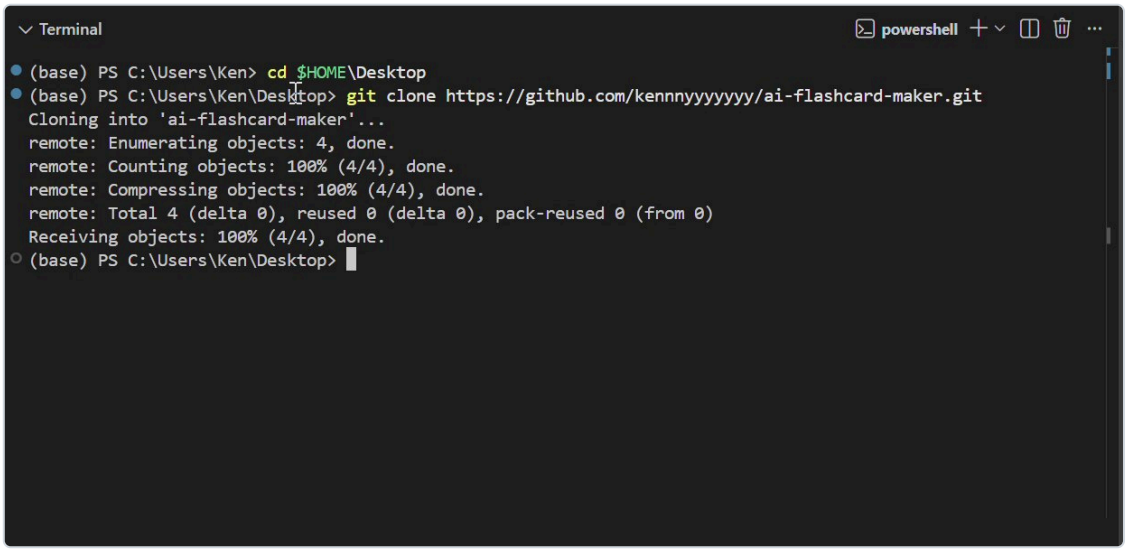


Screenshot 10: TRELLO_KEY and TRELLO_TOKEN stored as GitHub Actions secrets.

4. Add, edit, commit, push and clone

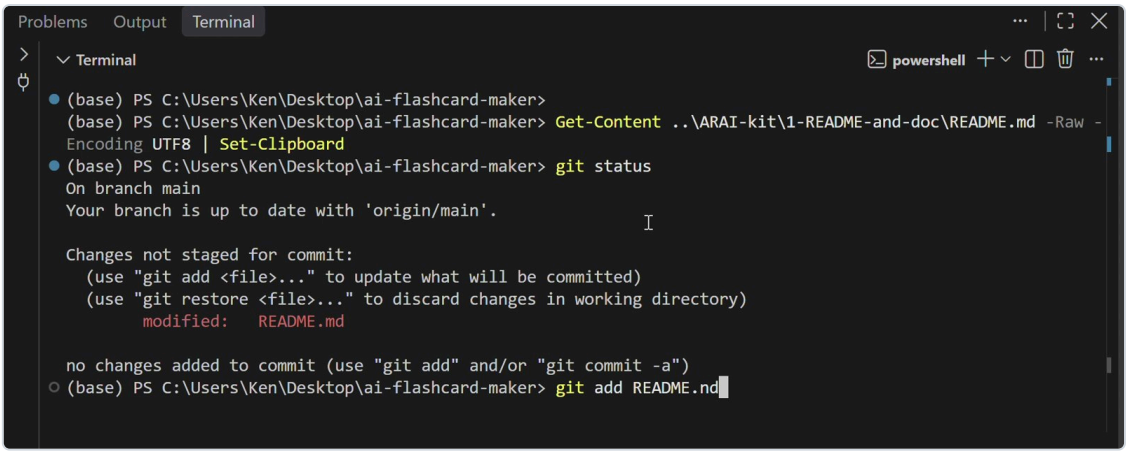
Clone. `git clone` downloads the whole repository, including its full history, and remembers GitHub as the remote called `origin`.

```
cd $HOME\Desktop
git clone https://github.com/kennnyyyyyy/ai-flashcard-maker.git
cd ai-flashcard-maker
```



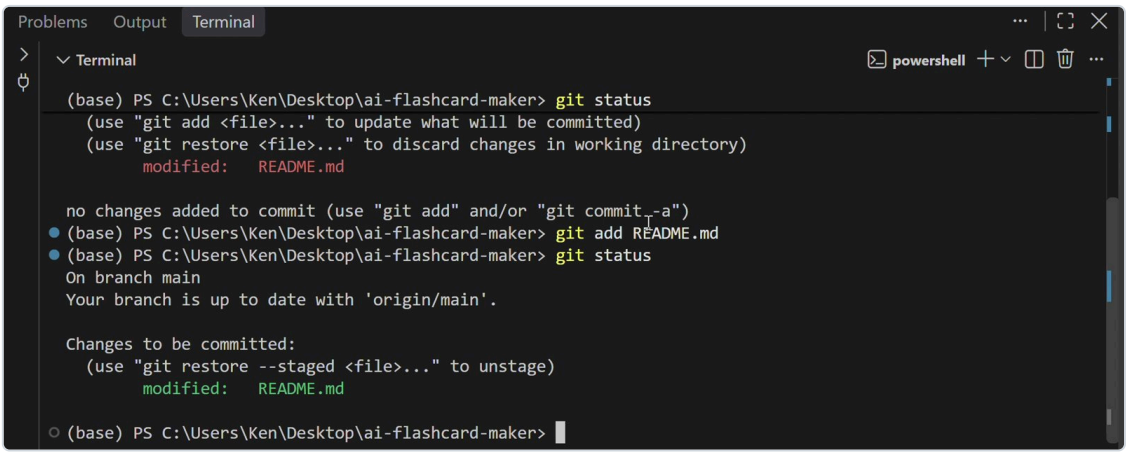
Screenshot 11: cloning the repository.

Edit. I replaced the default README with a full description of the project. After saving, `git status` shows `README.md` in red: Git has noticed the change, but it is not staged yet.



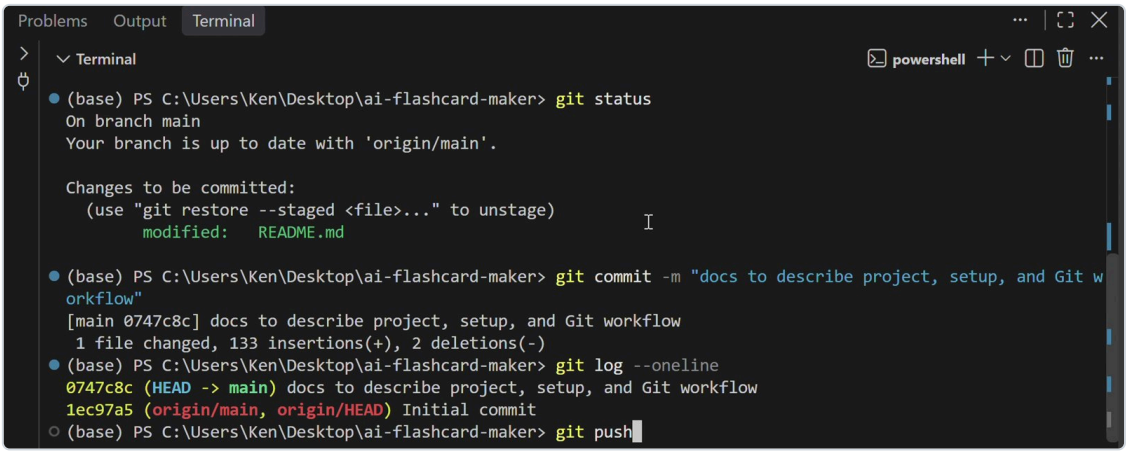
Screenshot 12: the edited README is "not staged for commit" (red).

Add. `git add README.md` puts the change in the staging area. Running `git status` again shows it in green under "Changes to be committed".



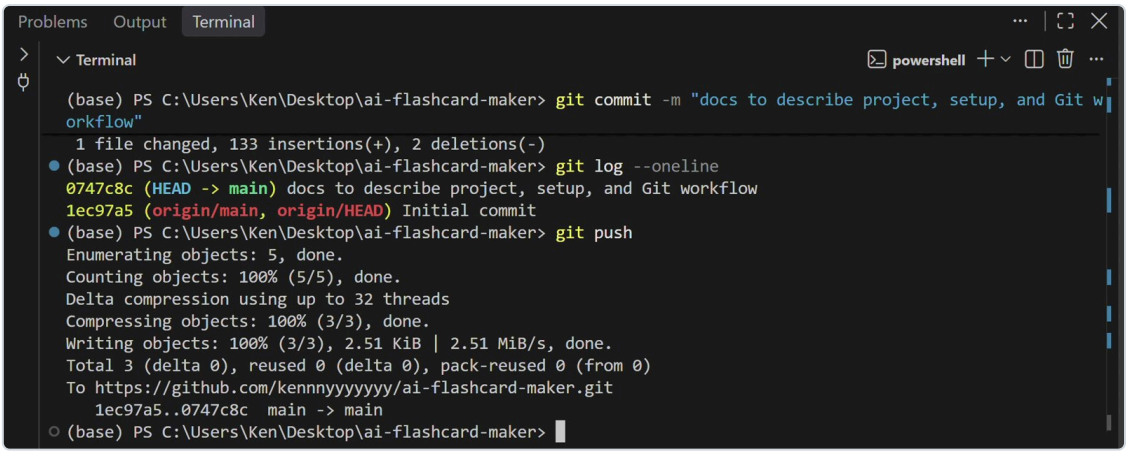
Screenshot 13: after git add, README.md is staged (green).

Commit. `git commit -m "..."` saves the staged change as a snapshot in my local history. `git log --oneline` shows the new commit (`0747c8c`) on top of the initial commit. At this point the commit exists only on my laptop: `origin/main` is still one commit behind.

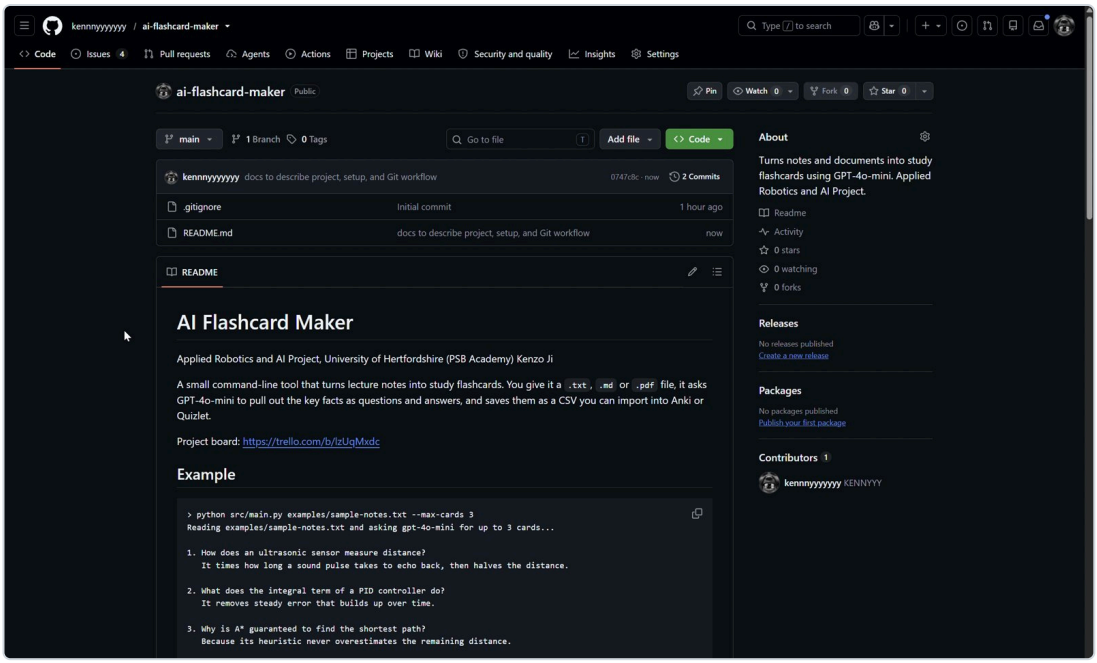


Screenshot 14: the commit and the log. Note that HEAD -> main is ahead of origin/main.

Push. `git push` uploads the commit to GitHub. The output ends with `1ec97a5..0747c8c main -> main`, which means GitHub's `main` now points to my new commit.



Screenshot 15: pushing the commit to GitHub.



Screenshot 16: the new README on GitHub after the push.

I then made a second commit the same way, adding `docs/git-add-commit-push.md`.

5. The difference between add, commit and push

A file moves through four places, and each command moves changes one step along:

```
working directory --git add--> staging area --git commit--> local repository --git push--> GitHub
```

git add chooses what goes into the next snapshot. It copies the current state of a file into the staging area, but nothing is saved to history yet. Because of this step, I can split my work into clean commits that each do one thing. In my branch I staged the project files first and the GitHub Actions files second, which produced two separate commits (screenshot 20). Files listed in `.gitignore`, such as `.env`, can never be staged, even with `git add .`.

git commit saves everything in the staging area as a permanent snapshot in my local repository. Each commit has a unique ID, my name, the time, and a message describing the change. Committing happens

entirely on my computer and needs no internet, so nobody else can see it yet. I use messages that start with the type of change, for example `chore:`, `ci:`, `feat:`, `test:` and `docs:`.

git push uploads the commits that GitHub does not have yet. After a push, the work is backed up and visible to collaborators, and GitHub Actions can test it. Pushing needs a network connection. If someone else had pushed first, I would `git pull` before pushing again.

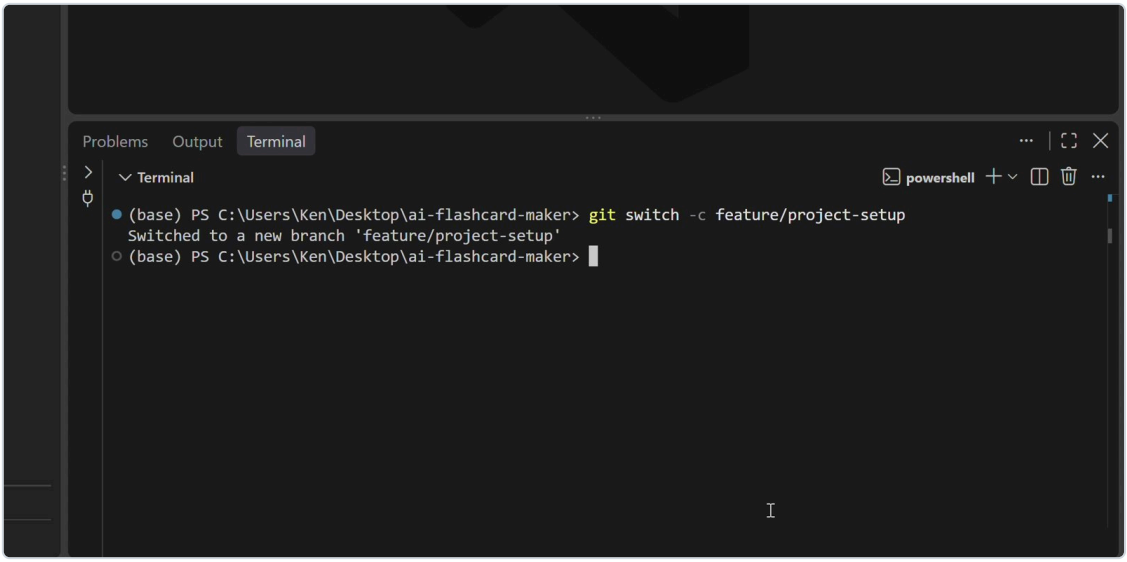
	git add	**git commit**	**git push**
Moves changes from	Working directory	Staging area	Local repository
Moves changes to	Staging area	Local repository	GitHub (remote)
Creates history?	No	Yes, one new commit	No, it copies existing commits
Needs internet?	No	No	Yes
Visible to others?	No	No	Yes
Easy to undo?	Yes (`git restore --staged`)	Yes, locally	Harder, others may have it

In short, **add** picks what to save, **commit** saves it on my computer, and **push** shares it with GitHub. Keeping them separate lets me make small, meaningful commits, work offline, and only share work once it is ready.

6. Branches and pull requests

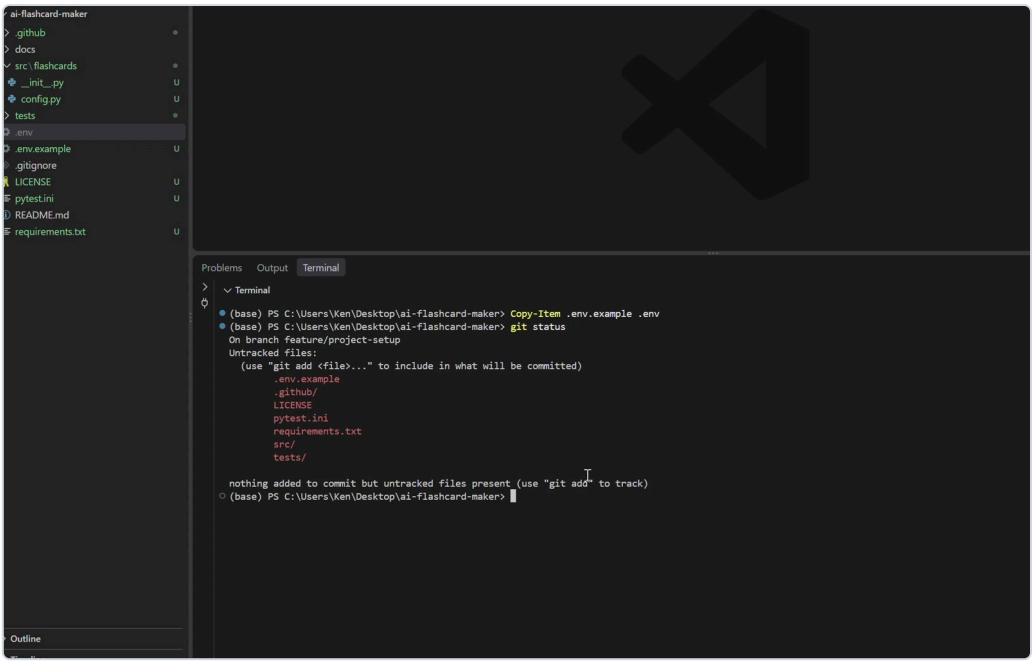
Branching strategy. `main` only ever holds working, tested code. Each Trello card / GitHub issue gets its own short-lived branch named after the work, for example `feature/project-setup`. Changes reach `main` only through a pull request, after the automated tests pass. The branch is then deleted.

Creating the branch. `git switch -c feature/project-setup` creates the branch and moves onto it.



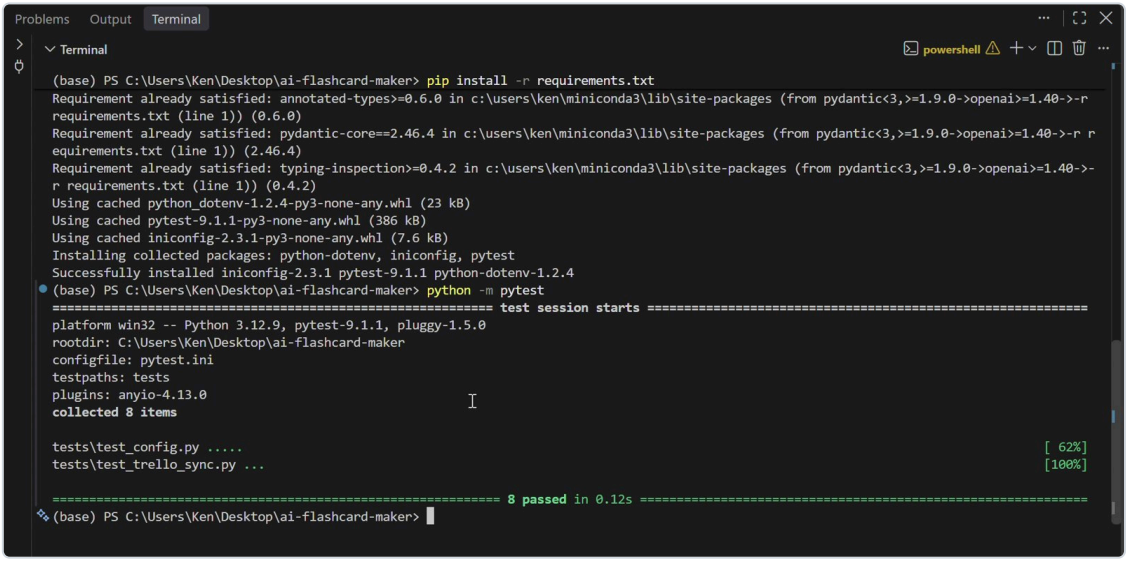
Screenshot 17: creating and switching to the feature branch.

On this branch I added the project setup files. My real `.env` file exists on disk, but `git status` only lists `.env.example`, because `.env` is ignored.



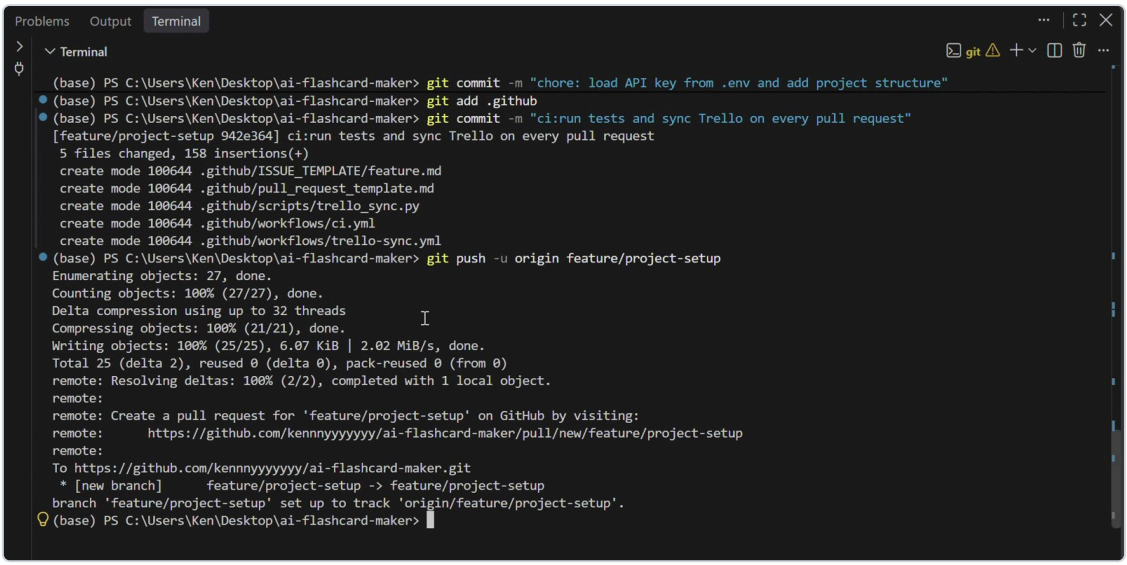
Screenshot 18: .env.example is listed as untracked, .env is not, even though it exists in the folder.

I ran the tests locally before committing.



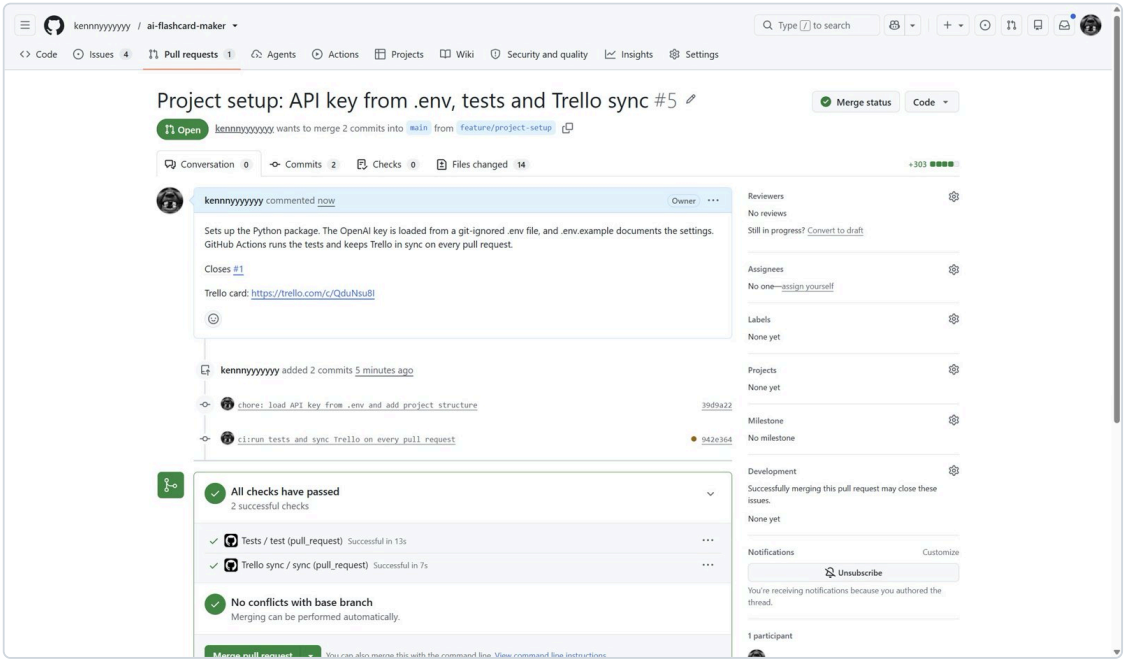
Screenshot 19: all 8 tests pass.

I committed in two small steps, then pushed the branch for the first time with `git push -u origin feature/project-setup`.

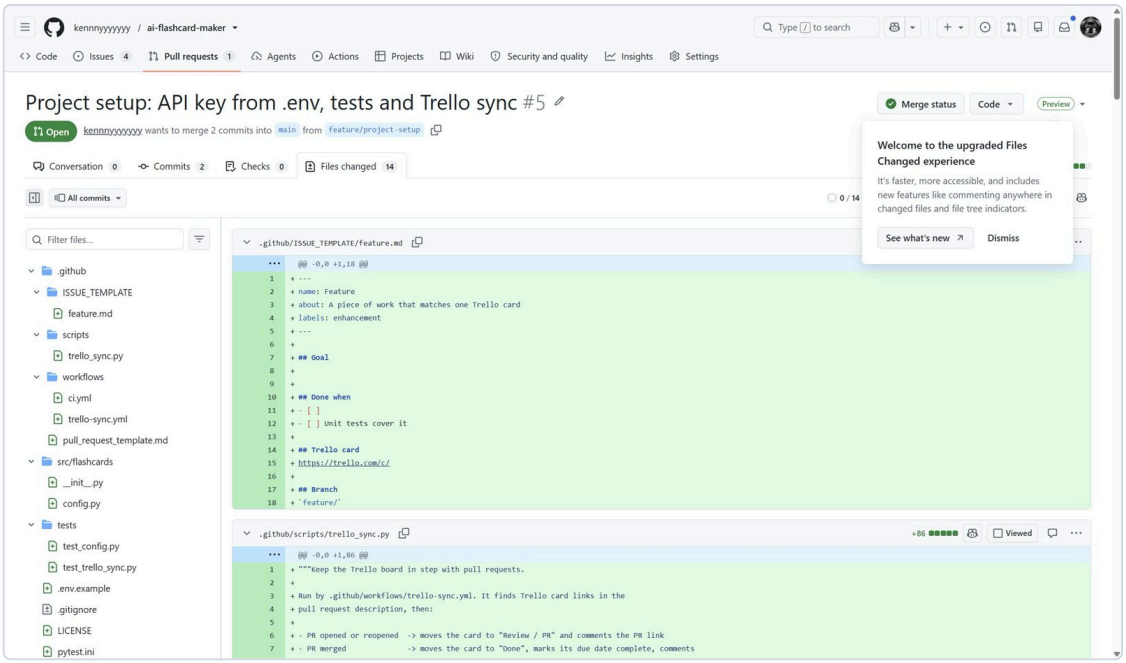


Screenshot 20: two separate commits, then pushing the new branch.

The pull request. I opened a pull request into `main`. The description explains the change, says `Closes #1`, and links the Trello card. GitHub Actions ran two checks automatically, the tests and the Trello sync, and both passed.

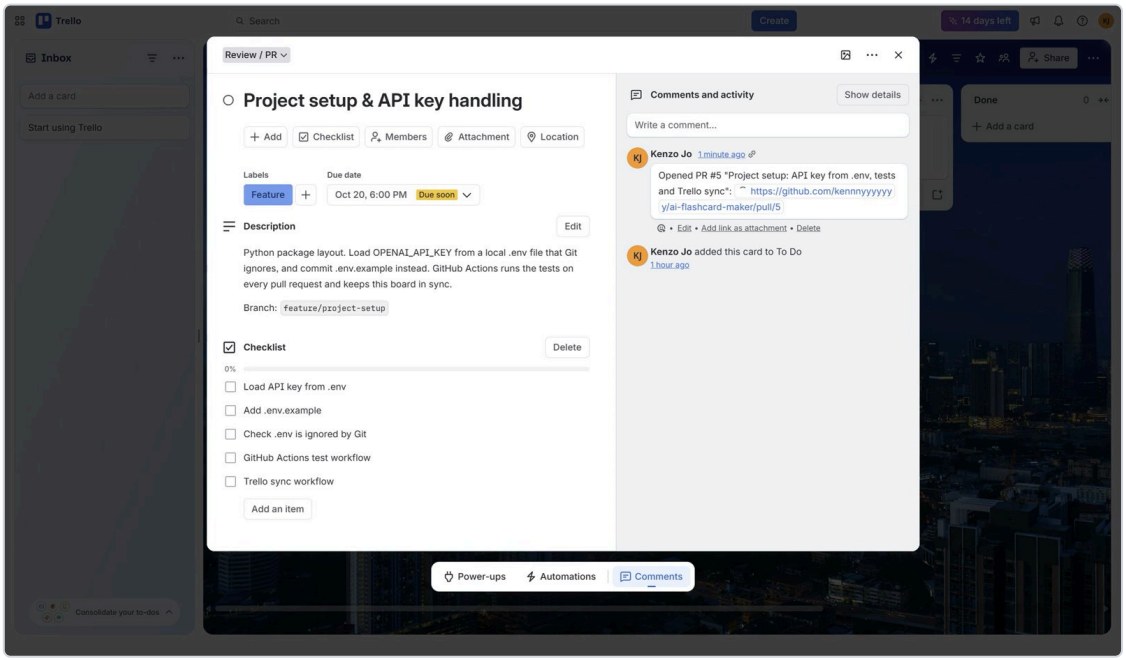


Screenshot 21: pull request #5 with both checks passing.

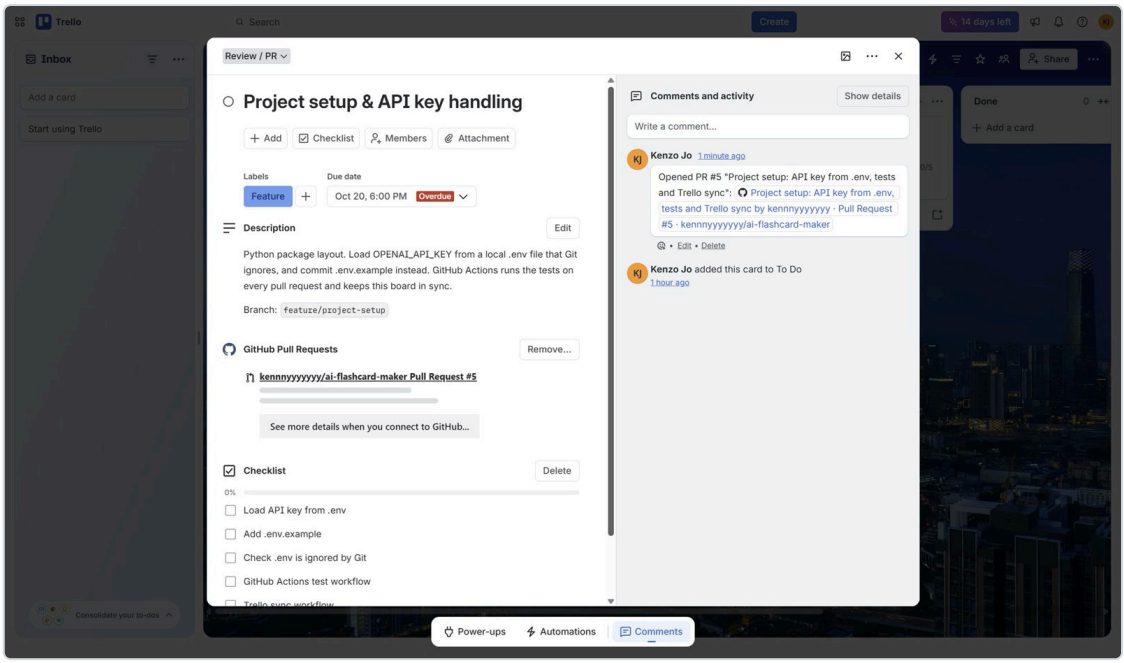


Screenshot 22: "Files changed" shows .env.example, but no .env and no API key.

At the same moment, the Trello sync action moved the card to **Review / PR** and commented the pull request link on it. I also attached the pull request to the card.

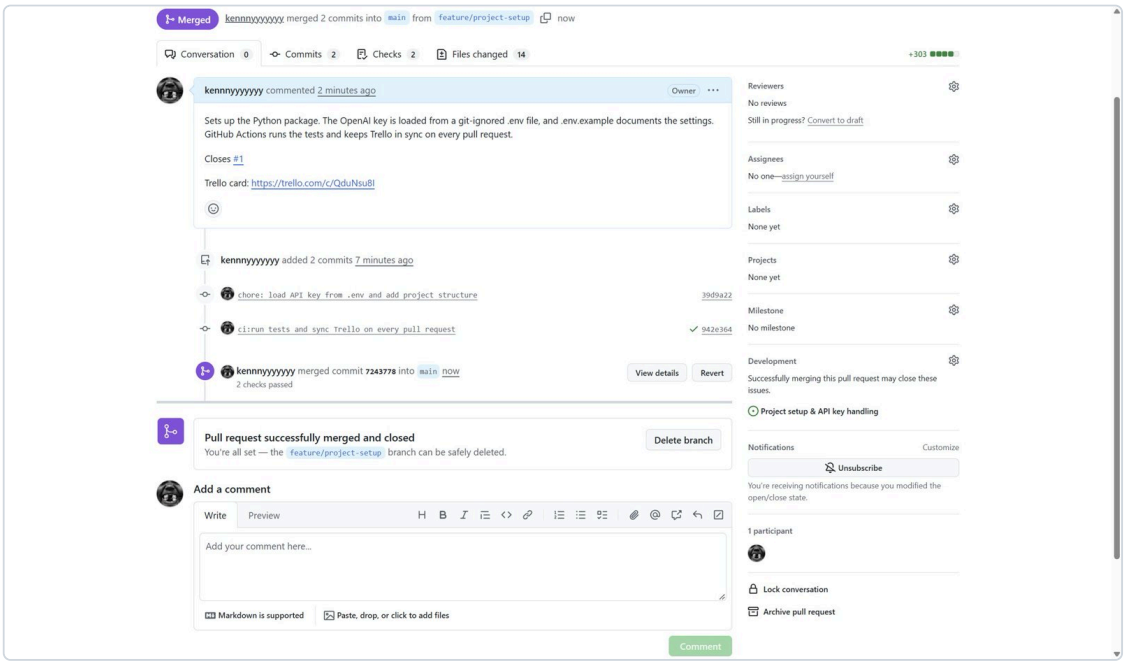


Screenshot 23: the card moved to Review / PR with the automatic comment.

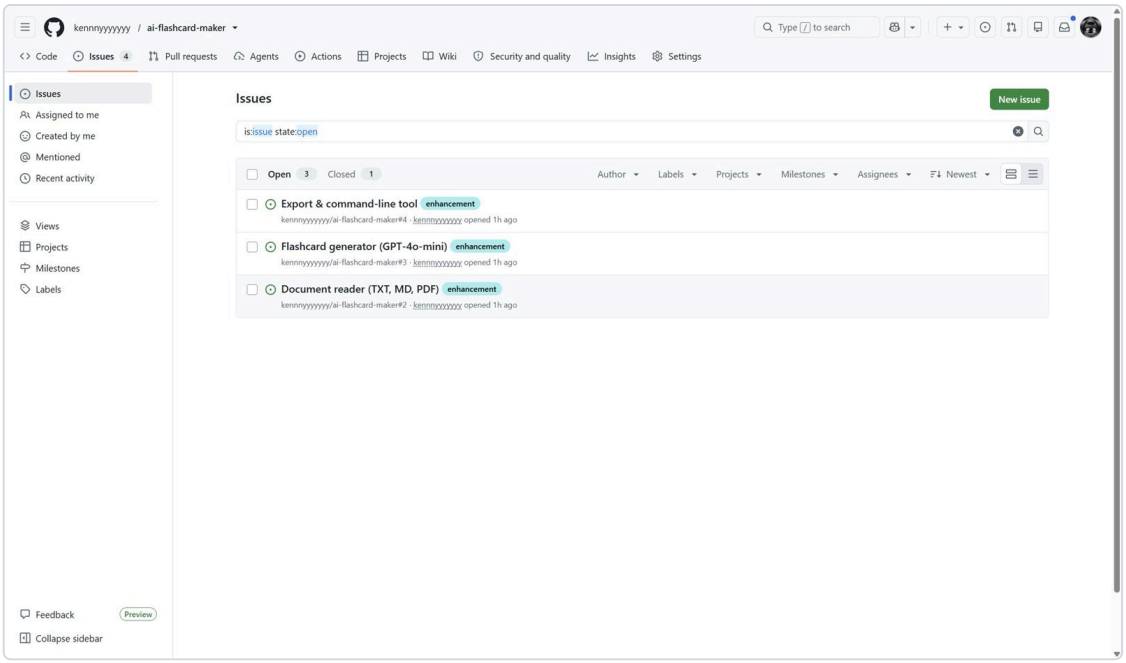


Screenshot 24: the pull request attached to the Trello card.

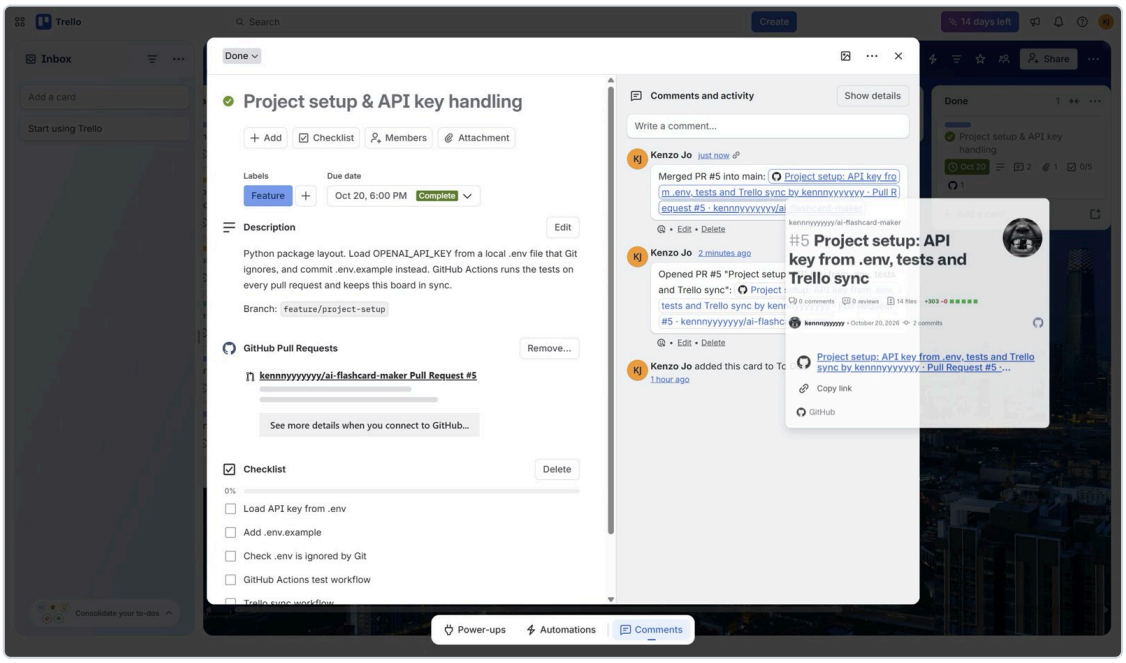
With the checks green, I merged the pull request and deleted the branch. Because the description said `Closes #1`, GitHub closed issue #1 automatically, and the Trello sync moved the card to **Done** and marked its due date complete.



Screenshot 25: pull request merged and branch deleted.



Screenshot 26: issue #1 closed automatically by the merge.



Screenshot 27: the card in Done with the "Merged PR #5" comment.

The other three features (document reader, flashcard generator, export and command-line tool) followed exactly the same cycle: card to In Progress, branch, small commits, push, pull request, tests, merge, card to Done.

7. How Trello and GitHub are connected

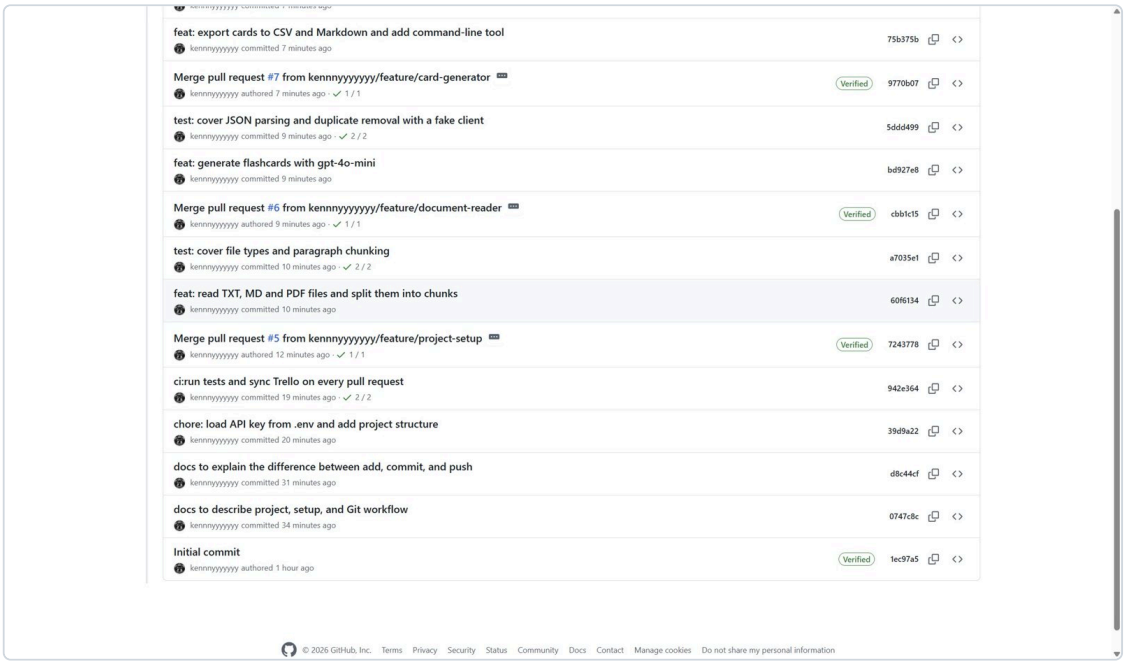
The board and the repository are linked in three ways:

1. **GitHub Power-Up.** Each pull request is attached to its Trello card.
2. **Issue and card links.** Every issue and pull request links its Trello card, and every pull request closes its issue with `Closes #n`.

3. **Trello sync GitHub Action** (`.github/workflows/trello-sync.yml`). When a pull request opens, it moves the linked card to Review / PR and comments the link. When the pull request is merged, it moves the card to Done, marks the due date complete, and comments the merge. It uses the `TRELLO_KEY` and `TRELLO_TOKEN` repository secrets, so no credentials are stored in the code.

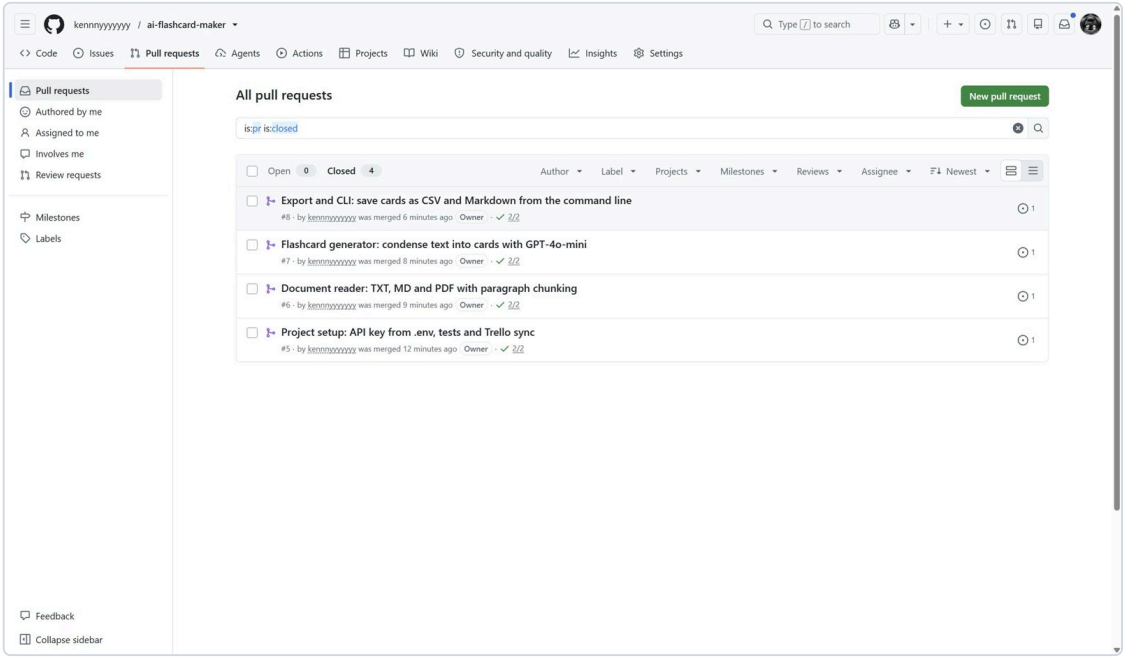
8. Evidence of continuous use

Commit history. Small, regular commits, each with a clear message, plus a merge commit for every pull request.



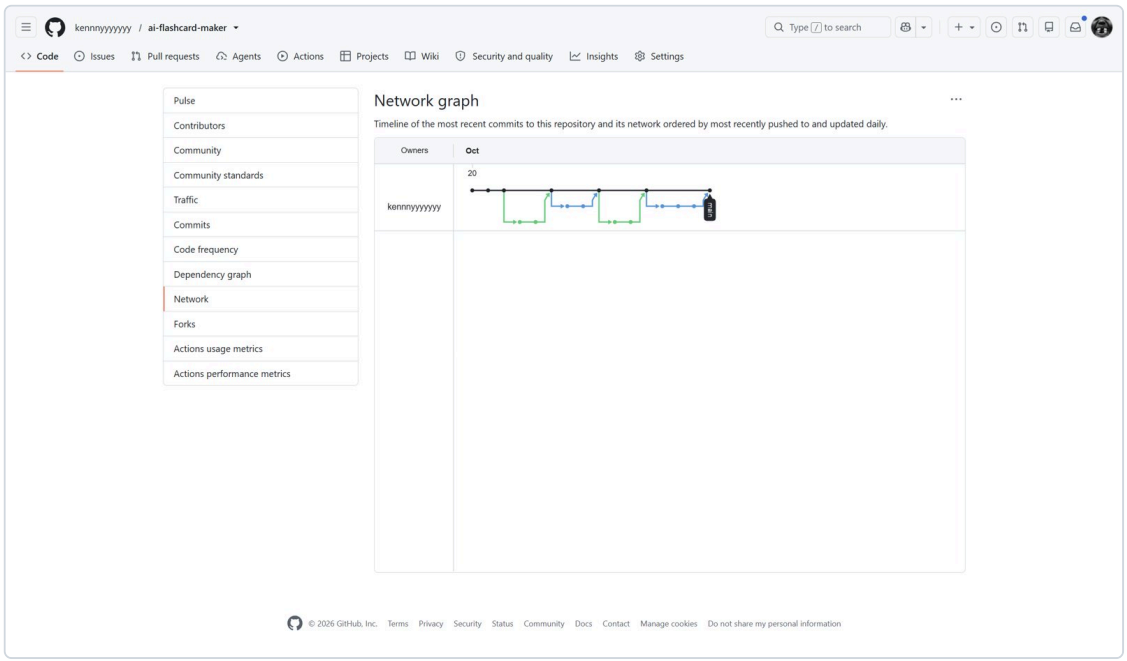
Screenshot 28: the commit history on main.

Pull requests. Every feature came in through its own pull request (#5 to #8). Each one passed its checks and closed its issue.

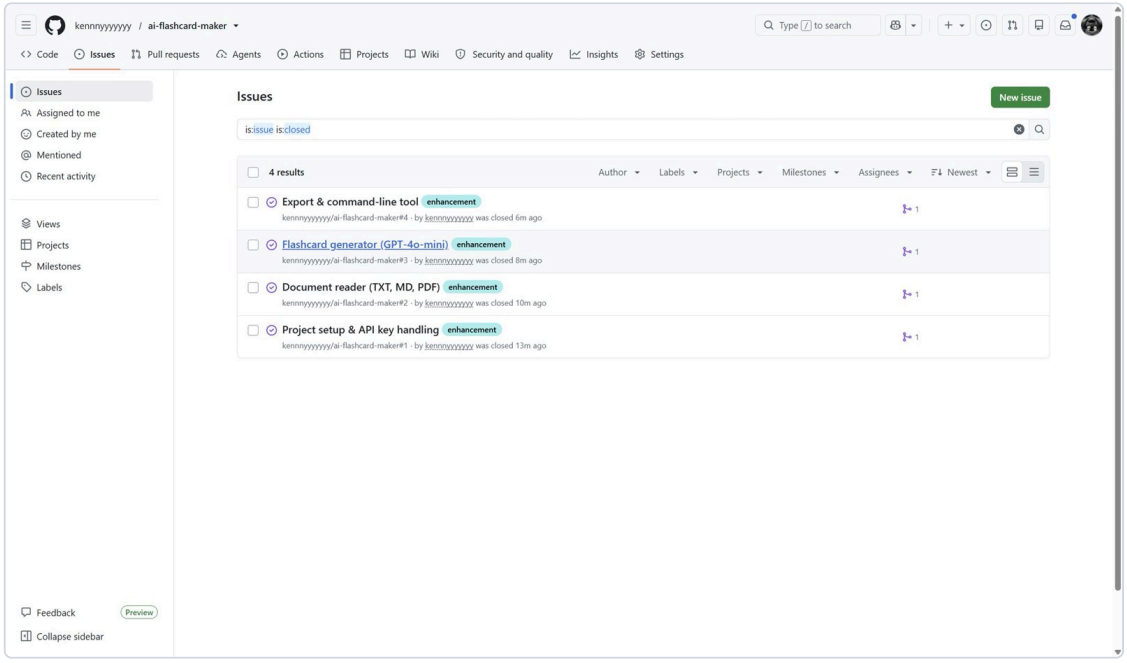


Screenshot 29: all pull requests merged.

Network graph. Each feature branch splits off from `main` and merges back in.

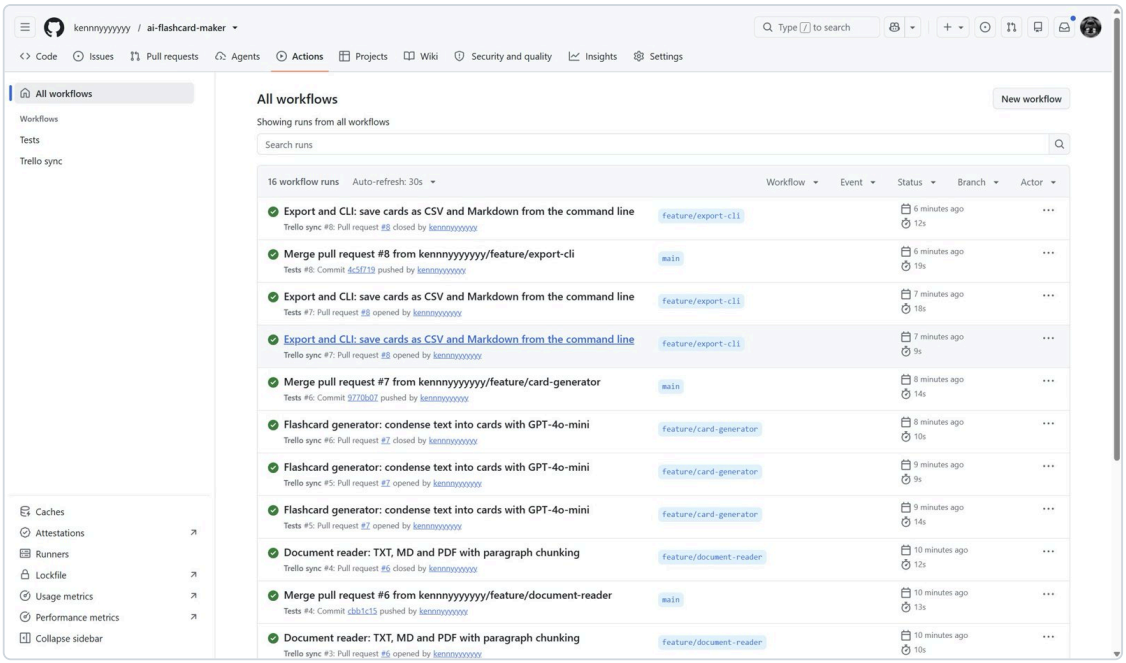


Screenshot 30: the branch and merge history.



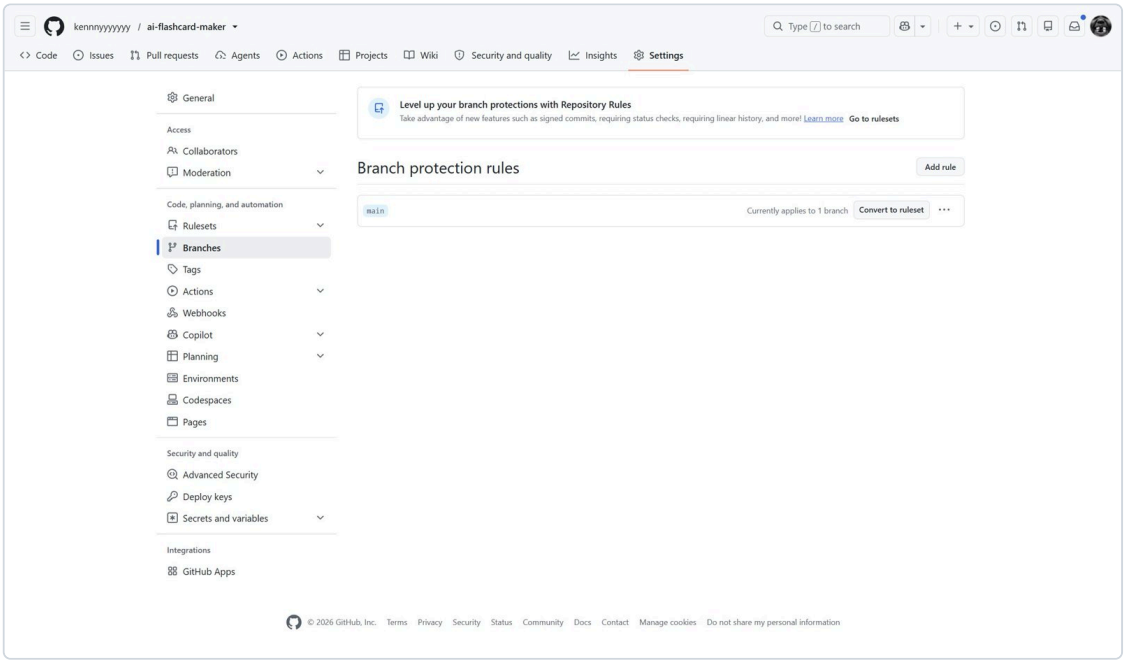
Screenshot 31: all four feature issues closed by their pull requests.

Automated checks. Every pull request ran the tests and the Trello sync.



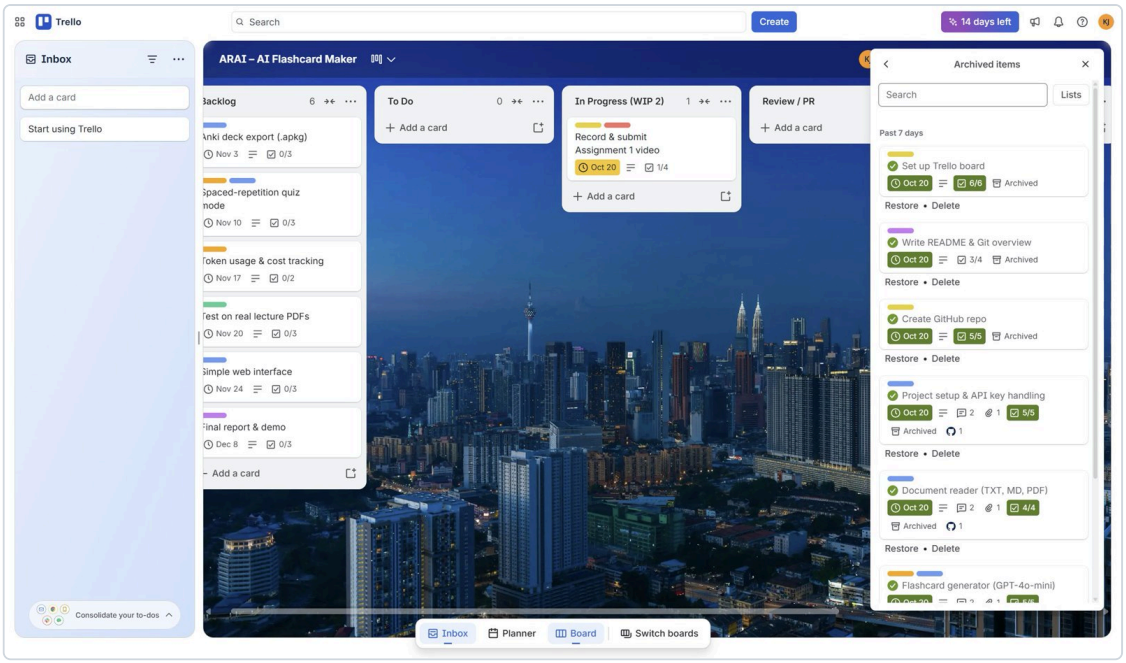
Screenshot 32: the GitHub Actions runs.

Branch protection. `main` is protected: a pull request and a passing `test` check are required before merging.



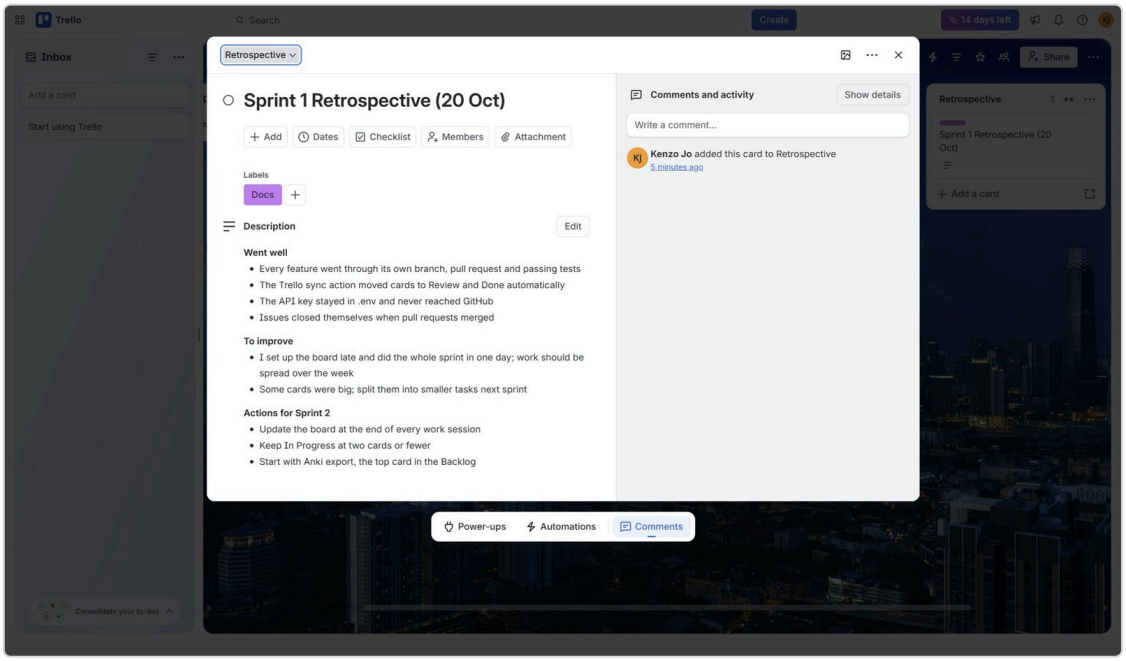
Screenshot 33: the protection rule on main.

Cards archived in Done with PR links. When the sprint ended, I archived every card in Done. The archived cards keep their pull request attachments and comments.



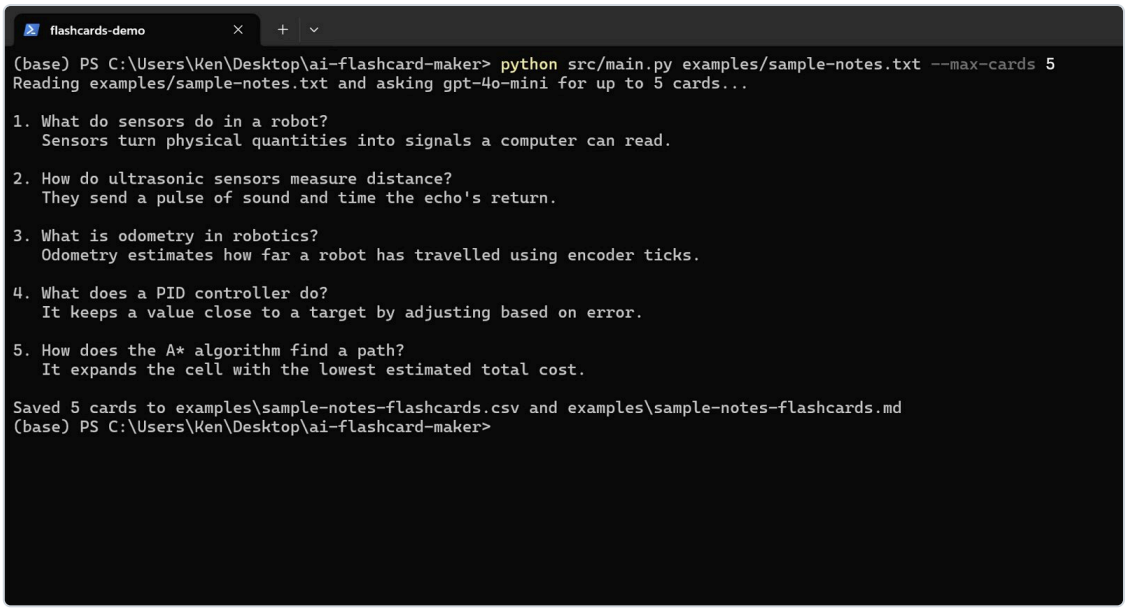
Screenshot 34: the archived Done cards.

Retrospective. At the end of the sprint I wrote a retrospective card covering what went well, what to improve, and actions for Sprint 2.

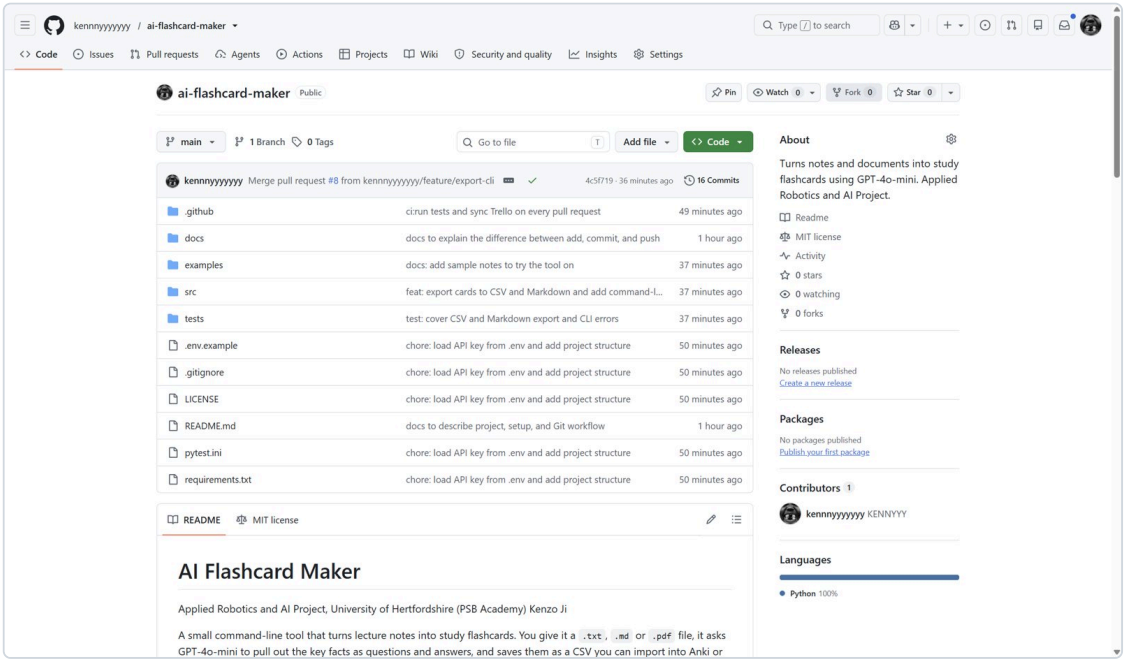


Screenshot 35: the Sprint 1 retrospective.

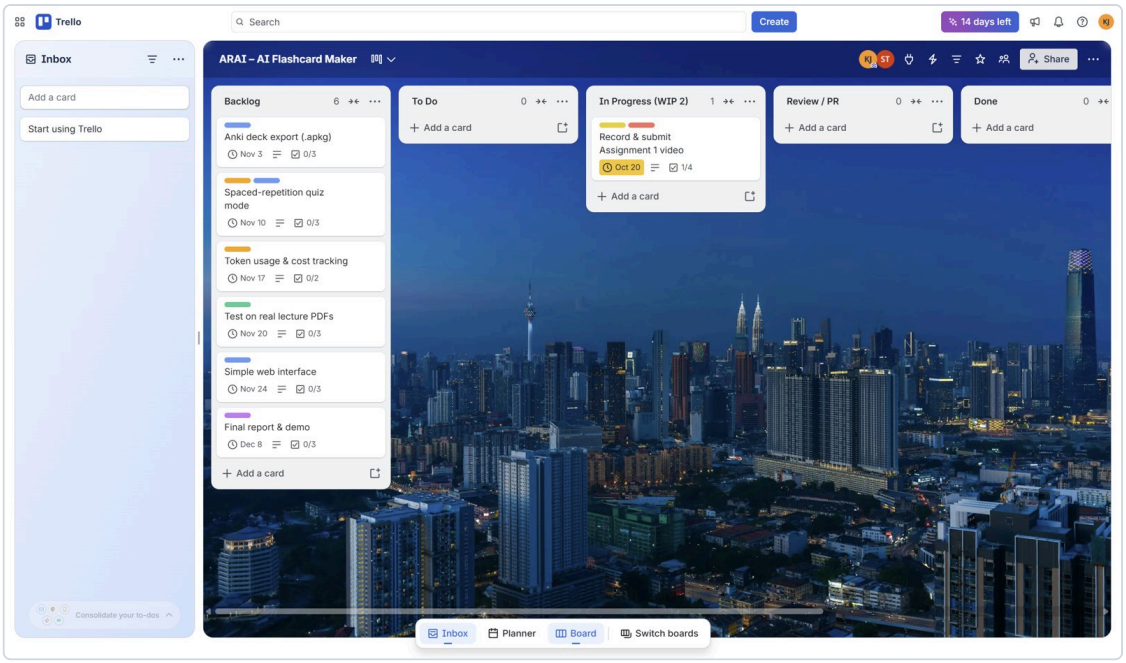
The finished tool. The result of the sprint, running for real: GPT-4o-mini turns a page of robotics notes into five flashcards.



Screenshot 36: a real run of the flashcard maker.



Screenshot 37: the repository today.



Screenshot 38: the board after the sprint. Done is archived, the video card is In Progress, the next sprint is planned in the Backlog, and the lecturer (ST) is a board member.

9. Summary

Trello is where I plan and track the work: every task is a card with a due date, a checklist and a branch name, and cards move through a workflow with a WIP limit. GitHub is where the work happens and is backed up: every change goes through add, commit and push, and every feature goes through a branch and a pull request with automated tests. The two are connected, so the board updates itself as pull requests open and merge. The commit history, pull requests, closed issues, archived cards and the retrospective together show the full lifecycle of the sprint, from planning to a working tool.